Mikhail Razakov

Compiler Infrastructure / Language Tooling Engineer

Moscow / remote · razakov.mikhail@outlook.com · Telegram · GitHub

PROFILE

I develop modular language and compiler infrastructure in C#/.NET and C++/LLVM. In UniversalToolchain, dependency resolution, component selection and construction of a verifiable LanguagePlan are separate from execution. I currently work on symbolic computation for SharpChecker at ISP RAS and previously completed an LLVM/C++ compiler internship at MCST.

TECHNICAL SKILLS

Compiler SDK / .NET: C#, .NET, LanguageCompiler, LanguagePlan, typed artifact contracts, dependency/conflict resolution, backend routing, Bytecode/AIR, interpreter/CIL

Compilers and IR: C++23, LLVM 22/IR, APInt, CFG/SSA, loop analysis and interprocedural effects; Rust, x86-64

Verification and tooling: Plan validation, deterministic pass ordering, regression/differential testing, LLVM Verifier, interpreter/CIL parity; Git, Linux, Python, CMake

EXPERIENCE

ISP RAS · SharpChecker · October 2026 — present

• Working on symbolic computation for C#/.NET static program analysis.

MCST · Compiler Engineering Internship · Jul–Aug 2026

• Worked on experimental LLVM LICM, checking instruction-hoisting legality against loop structure, memory accesses and side effects; compared before/after behavior using Python.

ENGINEERING PROJECTS

UniversalToolchain / Wist2

Developing modular language infrastructure. LanguageCompiler resolves dependencies and conflicts and builds an immutable LanguagePlan. A verifier checks typed artifact routes, backend ownership, contribution order and the canonical plan hash; runtime does not repeat semantic decisions. The pipeline includes frontend → Bytecode → AIR → optimizers → interpreter/CIL. Experimental PlanFuzz supports isolated replay and counterexample reduction but is not complete. The repository manifest records 1,324 expected test passes.

LLVM Interprocedural Global IV Optimization

An experimental LLVM pass promotes selected integer-global accesses in loops. Interprocedural effect analysis and legality checks are separated from IR rewriting. Passed 29/29 regression scenarios on LLVM 22.1.8, including structural checks, idempotence and conservative rejection cases.

x86-64 Compiler Backend & Register Allocation

Built an experimental backend for a restricted IR: SSA/CFG, liveness, register allocation, spills, phi lowering and SysV x86-64 code generation. An independent verifier checks register and stack-slot assignments; native execution results are compared against a reference IR interpreter.

EDUCATION

HSE University — Software Engineering

RECOGNITION

2026 · LangDev — UniversalToolchain: Presented the compiler infrastructure architecture on 9 October 2026. — Talk recording

2026 · HSE FCS · Open Source: UniversalToolchain was selected as one of the winners of the student open-source projects competition.

2025–2026 · MEPhI Junior: First-degree diplomas in the Information Technologies section in 2025 and 2026.

2026 · MCST LLVM/C++ selection: According to organizer correspondence: 5.0/5.0 and 104/104 official tests among 49 submitted solutions.